

Business Requirements Document (BRD):

Project Title: CoreConnect Messaging Platform

Technology Stack: Java (Spring Boot) & React.js

Status: Ready for Development

1. Project Goal

To build a functional, secure, and persistent messaging application. This project is designed to help junior developers master full-stack integration, CRUD operations, database relationships, and JWT-based security.

2. Scope of Requirements

2.1 Functional Requirements

- **Authentication:** Secure registration and login.
- **User Directory:** View a list of all registered users.
- **Persistent Messaging:** Send/receive text messages that remain available after page refresh.
- **Chat History:** Load previous messages between any two selected users.

2.2 Exclusions

- Real-time Online/Offline status indicators.
 - Typing indicators.
 - Multimedia attachments (Images/Video).
-

3. Technical Task List (Detailed Breakdown)

Phase 1: Backend (BE) - Java Spring Boot

Focus: Building the engine and security.

- **Task BE-1: Project Initialization & Dependencies**
 - Set up Spring Initializr with Web, JPA, Security, and PostgreSQL/H2.

- **Task BE-2: Database Schema Design**
 - Create `User` entity (id, username, password, email).
 - Create `Message` entity (id, senderId, receiverId, content, timestamp).
 - Establish `@ManyToMany` relationships between Messages and Users.
- **Task BE-3: Security Layer (JWT)**
 - Implement `UserDetailsService`.
 - Create JWT Utility class for generating and validating tokens.
 - Build `JwtFilter` to intercept and authorize incoming requests.
- **Task BE-4: API Development**
 - `POST /api/auth/register`: Create new user with BCrypt password encoding.
 - `POST /api/auth/login`: Authenticate and return JWT.
 - `GET /api/users`: Retrieve list of all users (excluding passwords).
 - `GET /api/messages/{contactId}`: Fetch conversation history between logged-in user and contact.
 - `POST /api/messages/send`: Save a new message to the database.

Phase 2: User Interface (UI) - Design & Layout

Focus: Creating a clean, professional user experience.

- **Task UI-1: Design System & Mockups**
 - Define a color palette (e.g., Primary: #075E54, Secondary: #ECE5DD).
 - Create a low-fidelity wireframe for the Login Page and the Dashboard.
- **Task UI-2: Component Architecture**
 - **Sidebar**: List of contacts with search bar.
 - **Chat Window**: Header (Contact Name), Message Area (Scrollable), and Input Bar.
 - **Message Bubble**: Distinct styles for "Sent" (Right/Green) vs "Received" (Left/White).
- **Task UI-3: Responsive Design**
 - Ensure the layout uses Flexbox or CSS Grid to handle different screen heights and widths.

Phase 3: Frontend (FE) - React Integration

Focus: Managing state and connecting to the API.

- **Task FE-1: Routing & Auth Guard**
 - Set up `react-router-dom`.
 - Create a "Protected Route" component that redirects unauthenticated users to `/login`.
- **Task FE-2: State Management (Context API)**
 - Create an `AuthContext` to store the JWT and user profile globally.
 - Implement `localStorage` logic to keep the user logged in on refresh.
- **Task FE-3: API Integration (Axios)**
 - Build an Axios instance with an **Interceptor** to automatically attach the `Authorization: Bearer <token>` header to every request.
- **Task FE-4: Messaging Logic**
 - Implement `useEffect` to fetch the contact list on dashboard load.
 - Implement a "Select Contact" function that triggers a fetch of the message history for that specific user.
 - Handle form submission for sending messages (Updating the local state immediately for a snappy UI feel).

4. Success Criteria

1. User can register and log in without errors.
2. Selecting a user from the sidebar correctly loads only the messages shared with that user.
3. Refreshing the browser does not log the user out.
4. Passwords are encrypted in the database (never stored in plain text).